

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Selecting and Planning the Software Development Methodology for Your Own Project

Software Development Methodologies

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 4
Output	Development methodology plan, selection worksheets, milestone map, online justification, and group presentation
Maximum Score	80 points

Student / Project Information

Block: <u>A1</u>	Date: <u>September 17, 2026</u>
System Title: Seaweed (Guso) Quality Classification Using Mobile Image Processing	Instructor: Jean Y. Villasin, MIT
Members: Zydane Angelo B. Adanza Timothy Josua C. Tavera Jherick Q. Peñalosa	

1. Activity Alignment and Purpose

Topic Learning Outcome: Compare common software development methodologies and select an appropriate approach for planning and managing the semester web-system project.

Course Outcome Alignment: Supports project planning and management by selecting and applying an appropriate development approach to the semester web-system project.

Syllabus Assessment: Development methodology plan; online methodology-selection justification.

Purpose of the Activity

Activities 2 and 3 established the architecture and technology stack of your semester project. In Activity 4, your team will decide how the project will be planned, developed, reviewed, and delivered. You will compare common methodologies, analyze project conditions, select a defensible primary approach, and map it to SDLC phases, iterations or sprints, and semester milestones. The final plan should become the team's working guide for managing development during the remaining laboratory activities.

Important distinction: SDLC phases describe the major categories of development work, while a methodology or development approach organizes how those phases are planned and repeated. Agile approaches do not remove requirements, design, testing, or deployment; they arrange work in adaptive cycles. DevOps and CI/CD are complementary delivery practices and do not replace the selected development methodology.

2. Intended Learning Objectives

- Explain the SDLC phases and the purpose of using a software development methodology.
- Distinguish predictive, iterative, incremental, adaptive, and hybrid development approaches.
- Compare Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, and Rapid Application Development (RAD).
- Compare Agile approaches including Scrum, Kanban, and Extreme Programming (XP).
- Explain how DevOps and CI/CD complement development and delivery without replacing the project methodology.
- Evaluate requirement stability, risk, scope, schedule, team capability, and stakeholder involvement when selecting a methodology.
- Map the selected methodology to SDLC phases, iterations, sprints, project milestones, reviews, and semester deliverables.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the comparison, selection, milestone planning, and oral defense.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.

Primary Output	One project-specific Development Methodology Plan for the semester web-system project.
-----------------------	--

Activity Item	Requirement
Supporting Output	Methodology comparison matrix, scenario analysis, selection-criteria worksheet, SDLC mapping, semester milestone plan, DevOps/CI/CD delivery plan, 120-180 word methodology-selection justification, and group presentation.
Recommended Tools	Word/Google Docs, spreadsheet or timeline tool, diagrams.net/Draw.io, Canva or PowerPoint, GitHub Projects/Trello or an equivalent task board, and the assigned methodology case-study reading.
Presentation Format	5-7 minutes per group followed by 2-3 minutes of questions. Every member must explain an assigned selection criterion, project stage/iteration, or milestone.

4. Pre-Laboratory Requirements

- Approved system title and a brief statement of the community or organizational problem being addressed.
- Final or revised web-system architecture diagram from Laboratory Activity No. 2.
- Final or revised technology stack blueprint from Laboratory Activity No. 3.
- List of major system features, primary users, stakeholders, and any required approvals or feedback points.
- Known project constraints such as semester deadlines, technical risks, external services, uncertain requirements, and team limitations.
- The assigned methodology case-study reading and lecture notes on SDLC phases and software development methodologies.
- Access to the group GitHub repository and any current backlog, task board, calendar, or milestone list used by the team.

5. Part A - Development Approach Classification

Before comparing individual methodologies, classify the planning pattern correctly. Use the descriptions below to distinguish how a project responds to requirements, risk, delivery, and change. A specific methodology may combine more than one pattern.

Category	Meaning
SDLC phases	Planning, requirements/analysis, design, development, testing, deployment, and maintenance/review activities that every project must address in some form.
Predictive approach	Plans most scope, sequence, and major deliverables early; works best when requirements are relatively stable and change can be controlled.

Iterative approach	Repeats development cycles to refine the solution, requirements, design, or implementation using learning from earlier cycles.
Incremental approach	Builds and delivers the system in usable pieces or feature increments rather than waiting for one complete final release.
Adaptive / Agile approach	Uses frequent planning, feedback, and change response through short cycles or continuous flow.
Hybrid approach	Combines clearly defined practices from more than one approach because different parts of the project have different needs.
DevOps / CI/CD	Complementary delivery practices that support integration, testing, release, deployment, feedback, and operational reliability; they are not substitutes for the project methodology.

Methodology Comparison Worksheet

Methodology / Approach	Primary Planning Pattern	Strong Fit / Advantage	Limitation / Risk
Waterfall	Predictive	Clear phase sequence and documentation; simple to plan.	Changes late in the semester can cause rework; weak fit when requirements evolve.
V-Model	Predictive	Strong relationship between development stages and testing.	Less flexible when requirements or implementation details change.
Prototyping	Iterative	Useful for discovering UI/workflow requirements through early prototypes.	Prototype work may be mistaken for the complete production system.
Iterative	Iterative	Allows repeated refinement based on testing and feedback.	Can become open-ended without clear time boxes and priorities.
Incremental	Incremental	Delivers usable features piece by piece.	Integration can become difficult if increments are not planned consistently.
Spiral	Iterative / Risk-driven	Strong focus on identifying and reducing major risks.	More process-heavy than needed for a small semester team.
Rapid Application Development (RAD)	Iterative / Adaptive	Fast construction using reusable components and strong user feedback.	Can sacrifice deeper analysis or quality if speed dominates.
Scrum	Adaptive / Iterative / Incremental	Short sprints, backlog prioritization, regular reviews, and working increments fit a small semester team.	Requires discipline in sprint scope, reviews, and task ownership.
Kanban	Adaptive / Continuous Flow	Flexible work queue and visual workflow; useful when priorities change frequently.	No fixed sprint boundary, which can make semester milestone planning less explicit.
Extreme Programming (XP)	Adaptive / Iterative	Strong engineering practices such as testing, refactoring, and frequent integration.	Some practices require close pair-programming and

			discipline that may be difficult to maintain.
--	--	--	---

6. Part B - Project Methodology Selection Criteria

Evaluate the conditions of your own project. Record evidence in the second column, then use the comparison cues to decide whether the condition favors a more predictive or a more iterative/adaptive approach. Your final choice should be based on the combined evidence, not on one criterion alone.

Selection Criterion	Your Project Evidence / Condition	More Predictive Tends to Fit When...	More Iterative / Adaptive Tends to Fit When...	Team Interpretation
Requirement stability	The system requirements are generally known, but UI details, workflow, and implementation choices may evolve after adviser/user feedback.	Fixed requirements support predictive planning.	Expected changes and feedback support iterative/adaptive work.	Favors Scrum because requirements can be kept in a prioritized backlog and refined between sprints.
Technical / project risk	The project includes image processing and MobileNetV3 integration, which may require repeated	Familiar technology with few unknowns favors predictive work.	ML behavior and integration uncertainty favor learning cycles.	Favors Scrum with risk-focused tasks and frequent technical validation.

	testing and tuning.			
Scope clarity	Major features are known: authentication, Guso classification, history/records, and responsive dashboard.	Clear boundaries favor predictive planning.	Feature details may change after prototypes and feedback.	Supports a stable product backlog with flexible sprint priorities.
Schedule / deadline	The project is a semester-based academic project with required presentations and deliverables.	Formal phase gates can fit fixed schedules.	Short time-boxes and usable increments are helpful before the final deadline.	Strongly favors Scrum sprints with measurable sprint goals.
Team size and experience	The project is developed by a small student team with shared responsibilities.	Specialized planned roles can fit larger/experienced teams.	Small cross-functional teams can collaborate and adjust tasks quickly.	Favors Scrum with shared ownership and clear task assignments.

Stakeholder involvement	Adviser/instructor and intended users can provide feedback during reviews and demonstrations.	Defined approval points favor predictive approaches.	Frequent feedback supports adaptive approaches.	Favors sprint reviews and backlog refinement.
Need for change and feedback	UI, classification workflow, and technical decisions may need improvement after testing.	Rare changes favor predictive planning.	Learning and feedback favor iterative/adaptive work.	Favors Scrum's inspect-and-adapt cycle.

Selection rule

Choose a methodology because it fits the project conditions, not because it is popular or familiar. A hybrid approach is acceptable only when the team identifies which practices are combined and where each one applies. DevOps and CI/CD may complement any selected methodology but should not be presented as the methodology itself.

7. Part C - Methodology Scenario-Analysis Workshop

For each scenario, identify a defensible methodology or approach and explain the tradeoff. More than one answer may be acceptable when the reasoning is supported by the project conditions.

8. Part D - Select and Map Your Project Methodology

Scenario	Key Project Condition	Best-Fit Methodology	Why It Fits	Main Risk / Limitation	Alternative / Hybrid Option
1. Government registration module	Waterfall	Forms, rules, and approval steps are fixed and can be planned early.	Late rule changes can require rework.	V-Model if strict verification is required.	Waterfall
2. New citizen reporting interface	Prototyping	Users are unsure of the best screen flow, so early prototypes can reveal usability issues.	Prototype may need to be refined repeatedly.	Scrum + Prototyping.	Prototyping
3. Verification-intensive transaction module	V-Model	Each development stage has a corresponding verification activity.	Less flexible if requirements change frequently.	Scrum with strong test gates can also work.	V-Model
4. Semester project with frequent adviser feedback	Scrum	Short sprints, working features, reviews, and reprioritization fit a small student team.	Requires regular participation and disciplined sprint planning.	Scrum + Kanban-style task board.	Scrum
5. Short deadline for an early working demo	RAD	Rapid construction and reusable components support an early usable demo.	Speed can cause technical debt or insufficient testing.	Scrum + Prototyping/RAD practices.	RAD
6. GIS/ML feature with uncertain integration risk	Spiral	Risk-focused cycles are useful when model/API behavior and data quality are uncertain.	Risk analysis can become process-heavy.	Scrum + explicit ML risk spikes/technical experiments.	Spiral

Scenario-analysis checklist:

- Use evidence from requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Do not select a methodology only because the team already knows it or because it is currently popular.
- Distinguish the development methodology from tools such as Trello/GitHub Projects and from complementary DevOps/CI/CD practices.
- State at least one tradeoff, limitation, or alternative approach for every scenario.

Selected primary methodology: Scrum

Complementary approach / practice(s): Adaptive / Agile, with iterative and incremental delivery

Why this choice fits the project (3-5 sentences):

The reason why this choice fits the project (3–5 sentences): Scrum fits our small semester project because we need working features regularly, feedback from our adviser, and flexibility when technical or UI details change. The Guso classification feature also involves ML/image-processing risk, so short cycles allow the team to test the model and integration early instead of discovering problems near the deadline. Each sprint can produce a usable increment such as authentication, image classification, history, or dashboard improvements. Regular sprint reviews help the team inspect progress and adjust the next sprint while keeping the overall semester scope controlled.

Project Methodology and SDLC Mapping Worksheet

Map the major SDLC activities to the way your selected methodology will actually organize the semester project. If the methodology repeats phases, show where the repetition occurs. If it uses sprints or continuous flow, identify the planning and review points clearly.

SDLC Phase / Activity	How It Is Handled	Time-Box / Duration	Iteration Sprint / Stage	Main Team Role(s)	Stakeholder Check	Deliverable / Exit Evidence
Project planning	Define product goal, scope, team responsibilities, backlog, risks, and semester milestones.	1 week initially; refined each sprint	Sprint 0 / ongoing backlog refinement	Whole team / coordinator	Adviser confirms project direction.	Prioritized backlog + sprint plan.
Requirements / analysis	Identify user needs, acceptance criteria, feature priorities, and changes.	1 week initially; refinement every sprint	Sprint 0 + every sprint	Whole team / requirements lead	Adviser/user feedback.	Approved/refined requirements and acceptance criteria.
Design	Design UI, API flow, database mapping, and classification workflow; prototype when needed.	1 week initially; refined per sprint	Sprint 0 + each relevant sprint	UI/backend/database members	Review prototype/architecture.	Updated design/prototype and technical notes.
Development / implementation	Build features in small increments and integrate Flutter Web, FastAPI, PostgreSQL, and MobileNetV3.	2-week sprints	Sprints 1–5	Developers	Sprint review/demo.	Working feature increment committed to GitHub.
Testing / verification	Unit, API, database, UI, image-validation, and classification tests are performed continuously.	Throughout each sprint	Every sprint	Developers + tester/reviewer	Review test evidence and defects.	Test results, fixed defects, regression evidence.
Integration / deployment	Integrate branches, build the web client/API, run checks, and deploy to test/staging then final environment.	Each sprint; final release stage	Sprint 2 onward	Backend/deployment lead + team	Deployment/demo check.	Successful build/deployment and smoke test.

Review / feedback	Sprint review/demo followed by feedback and backlog adjustment.	End of every 2week sprint	Every sprint	Whole team + adviser	Adviser/stakeholder review.	Feedback record + updated backlog.
Maintenance / improvement	Fix defects, improve model/inference performance, refine UI, and	Final sprints and post-demo	Sprints 4–5 / final	Whole team	Final acceptance/demo.	Release candidate, defect list, final improvements.
	prepare final release.					

SDLC mapping guidance

Waterfall and V-Model usually emphasize planned phases and formal transitions. Prototyping, Iterative, Incremental, Spiral, and RAD repeat or accelerate selected activities. Scrum normally organizes work into sprints; Kanban uses continuous flow and work-in-progress control; XP emphasizes short feedback cycles and engineering practices. Your mapping must reflect the actual method selected by the team.

9. Part E - Semester Project Milestone Plan

Translate the selected methodology into a realistic semester schedule. Every milestone should have a measurable output and a review, approval, demonstration, test, or feedback point. Use your actual course deadlines when they are already known.

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Week 1 Oct 5 – Oct 11	Sprint 0 – Planning & Requirements	System scope; user stories; acceptance criteria; prioritized backlog.	Review Activity 2–3 architecture/stack; confirm requirements; identify risks.	Requirements list, backlog, sprint plan.	Adviser/instructor review.
Weeks 2–3 Oct 12 – Oct 25	Sprint 1 – System Foundation	Flutter Web structure, routing, login/register UI, FastAPI project setup, PostgreSQL connection.	Set up repository/branches; API foundation; database connection; basic client-server test.	Working login/register foundation + API health check.	Sprint review and feedback.
Weeks 4–5 Oct 26 – Nov 1	Sprint 2 – Classification Core	Image upload, server-side validation, image processing, MobileNetV3 integration.	Implement classification endpoint; test valid/invalid images; integrate model.	Working Guso classification prototype + test evidence.	Adviser demo; collect classification/UI feedback.

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Weeks 6–7 Nov 2 – Nov 8	Sprint 3 – Records & Dashboard	Classification result display, history/records, userspecific access, responsive dashboard.	Connect PostgreSQL records; Riverpod state updates; implement history and record views.	End-to-end classification + history workflow.	Sprint review / acceptance feedback.
Weeks 8–9 Nov 9 – Nov 15	Sprint 4 – Quality & Integration	Validation improvements, error handling, security, UI/UX refinement, performance improvements.	API/UI & UX/database tests; JWT checks; integration testing; ML latency checks.	Integrated test build + defect log + resolved issues.	Adviser review and reprioritization.
Weeks 10–11 Nov 16 – Nov 22	Sprint 5 – Final Testing & Fixes/Additional	Final integrated system, bug resolutions, minor tweaks, and final touches.core features, (download/save record) deployment preparation.	Thorough testing of the complete system, resolving lingering bugs, implementing minor additions and documentation.	Fully stable, polished application ready for final presentation..	Final review / approval for presentation.
Week 12 Nov 23 – Nov 28	Final Presentation	Final system, documentation, presentation materials.	Final acceptance testing, fixes, repository cleanup, final demonstration.	Final working system + methodology artifacts + presentation.	Final adviser/instructor evaluation.

Milestone planning rule

The methodology plan must be visible in the schedule. A sprint-based plan should show sprint goals and reviews; an incremental plan should show usable feature increments; a predictive plan should show phase gates and approval points; a Kanban-style plan should show how work enters, moves through, and exits the workflow. Do not use vague milestones such as “continue coding” without an observable deliverable.

10. Part F - DevOps and CI/CD Complementary Delivery Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Version-control workflow	Use Git/GitHub with main branch and feature branches. Commit focused changes, open pull requests, and review before merging.	PR/merge history, meaningful commits, reviewed changes.
Continuous integration	When a branch/PR is opened or updated, run formatting, static checks, tests, and build checks for the affected client/backend code.	Passing CI/check results before merge.

Testing practice	Run unit tests for logic, API tests for FastAPI endpoints, database tests for important queries/constraints, UI checks, and end-to-end classification tests.	Test reports/screenshots/logs and defect records.
Database change control	Store PostgreSQL migrations/scripts in the database folder and review schema changes with application changes.	Migration files, reviewed SQL, successful migration test.
Deployment stages	Use local development first, then test/staging where available, then final hosting for Flutter Web/FastAPI/PostgreSQL.	Deployment logs and smoke-test results.
Environment configuration	Keep API base URLs, database credentials, JWT secrets, and other sensitive values in environment configuration; never commit real secrets.	.env.example plus configured deployment variables.

of

Monitoring / feedback	Record API errors, defects, model issues, stakeholder feedback, and completed work in the project workspace/backlog.	Issue/feedback records linked to backlog tasks.
Release / rollback	Tag or identify stable releases; keep previous working deployment/configuration available so a defective release can be reverted.	Release tag/notes and rollback procedure/test.

Recommended minimum methodology evidence in the project repository or shared workspace (adapt as needed):

```
docs/methodology/ development-methodology-
plan.pdf methodology-selection-justification.md
```

```

milestone-plan/ sprint-or-
iteration-notes/ review-
and-feedback-records/ test-
evidence/ release-notes/
README.md - brief description of the team's development workflow

```

11. Part G - Construct the Development Methodology Plan

Create one clear project-specific methodology plan that explains how your team will manage the semester project from requirements through delivery. The plan may be presented as a structured one-page or multi-page visual, timeline, workflow, or combination of diagram and schedule, but it must remain consistent with the completed worksheets.

No.	Required Element	What Must Be Shown
1	PROJECT IDENTITY:	Seaweed (Guso) Quality Classification Using Image Processing. The system is a web-based ML application for Farmers and Sellers that classifies Guso quality from images and provides records/history.
2	PRIMARY METHODOLOGY:	Scrum – Adaptive/Agile, using iterative and incremental delivery.
3	PROJECT CONDITIONS:	The project has a semester deadline, a small student team, adviser feedback, known major features but some evolving details, and technical uncertainty around image processing and MobileNetV3 integration.
4	SPRINT STRUCTURE:	Two-week sprints after a short planning/foundation stage. Each sprint follows: Backlog refinement → Sprint planning → Design/development → Testing/integration → Sprint review → Retrospective → Next sprint.
5	BACKLOG CONTROL:	Features are recorded as user stories/tasks with priority and acceptance criteria. Changes are reviewed during backlog refinement; urgent changes are accepted only when they are justified and the sprint scope/deadline impact is understood.
6	TEAM RESPONSIBILITIES:	Members share planning and review. Tasks are assigned for Flutter/UI, FastAPI/API, PostgreSQL/data, ML/image processing, testing, documentation, and deployment, with peer review before important merges.
7	STAKEHOLDER INVOLVEMENT:	Adviser/instructor feedback is collected during sprint reviews and demonstrations. User feedback is used to refine the backlog without changing the core scope without team/adviser agreement.
8	QUALITY:	Testing occurs in every sprint. Defects become backlog items, are prioritized, fixed, and regression-tested before release.

9	ADAPTATION RULE:	If requirements change, the team records the change, evaluates impact on scope/time/technical risk, and either reprioritizes the backlog for the next sprint or obtains approval before changing a committed milestone.
---	------------------	---

Minimum Plan Rule

Your plan must show a complete management path from project requirements or backlog -> planned phase/iteration/sprint/work queue -> development and testing -> review/feedback -> milestone or release. If the selected method is iterative or adaptive, show how the cycle repeats. If it is predictive, show phase gates. If it is hybrid, label exactly where each approach is used.

The example below illustrates the decision-and-planning logic expected in the final output. Replace the generic labels with the actual constraints, methodology, iteration/stage structure, milestones, stakeholder reviews, and delivery practices of your own project.

Reference Methodology Selection and Semester Planning Flow

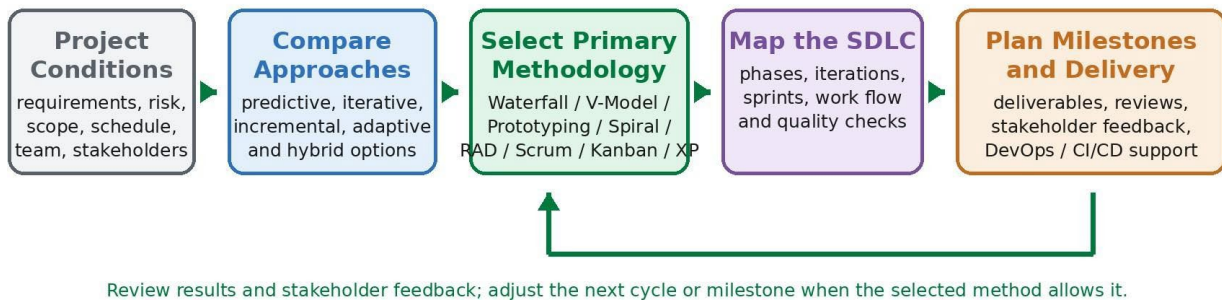


Figure 1. Reference methodology selection and semester planning flow.

Step	Reference Flow
1	Identify project conditions: requirement stability, risk, scope, schedule, team, and stakeholder involvement.
2	Compare predictive, iterative, incremental, adaptive, and hybrid options using the actual project evidence.
3	Select one primary methodology and identify any complementary practices such as prototyping, DevOps, or CI/CD.
4	Map SDLC phases to stages, iterations, sprints, or continuous workflow according to the selected methodology.
5	Define semester milestones, deliverables, testing points, demonstrations, reviews, and stakeholder feedback.
6	Use results and feedback to continue the planned next stage or adapt the next cycle when the selected methodology permits it.

13. Part H - Methodology-to-Project Trace

Selected feature / work package:

Target milestone / release: _____

Step / Iteration	Planned Work	Methodology Principle / Practice	Evidence / Deliverable	Review / Feedback
Sprint 1	Prepare Flutter Web upload interface, FastAPI classification endpoint skeleton, PostgreSQL connection, and authentication foundation.	Incremental delivery; establish a small usable foundation before the ML feature.	Working client-to-API connection and database/API health check.	Sprint review; adviser confirms workflow.

Sprint 2 – Planning	Define classification acceptance criteria and image-validation rules.	Backlog refinement and sprint planning.	Approved user story, acceptance criteria, and test cases.	Team review before implementation.
Sprint 2 – Development	Implement image upload, server-side validation, preprocessing, and MobileNetV3 inference.	Iterative development; integrate and test risky technical work early.	Classification endpoint successfully processes valid test images.	Technical review and test evidence.
Sprint 2 – Testing	Test valid images, invalid files, API errors, authentication,	Continuous testing and feedback.	Test results and corrected defects.	Team/adviser review.

	and classification response.			
Sprint 2 – Review	Demonstrate classification result in Flutter Web and collect feedback.	Sprint review / inspect and adapt.	Working classification prototype + feedback record.	Adviser/user feedback.
Next Sprint	Refine UI, accuracy-related workflow, error handling, history integration, and performance based on results.	Adapt next sprint backlog using evidence and feedback.	Updated backlog and improved classification feature.	Next sprint planning.
Sprint 1	Prepare Flutter Web upload interface, FastAPI classification endpoint skeleton, PostgreSQL connection, and authentication foundation.	Incremental delivery; establish a small usable foundation before the ML feature.	Working client-to-API connection and database/API health check.	Sprint review; adviser confirms workflow.
Sprint 2 – Planning	Define classification acceptance criteria and image-validation rules.	Backlog refinement and sprint planning.	Approved user story, acceptance criteria, and test cases.	Team review before implementation.
Sprint 2 – Development	Implement image upload, server-side validation, preprocessing, and MobileNetV3 inference.	Iterative development; integrate and test risky technical work early.	Classification endpoint successfully processes valid test images.	Technical review and test evidence.

14. Part I - Methodology-Selection Justification

We selected Scrum as the primary development methodology because our Guso Quality Classification project has evolving requirements, ML integration risks, a semester deadline, and a small team that needs frequent feedback. Scrum is an adaptive and iterative approach that allows us to divide the system into short sprints and deliver usable features regularly. This is more suitable than Waterfall because requirements and technical details may change after adviser or user feedback, while Waterfall relies more heavily on early planning. It is also more suitable than Kanban for our semester project because fixed sprints give us clear goals, review points, and measurable deliverables for each laboratory milestone. We will organize requirements into a prioritized backlog, then repeat planning, development, testing, review, and improvement in each sprint. GitHub, automated checks, testing, controlled database migrations, and staged deployment will complement Scrum through DevOps/CI practices. A key risk is limited time for repeated rework, so sprint scope must be controlled.

Checklist A	Checklist B
☐ The official project title, users, features, and real project conditions are used.	☐ The selected primary methodology is clearly named and correctly classified.
☐ Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP were compared.	☐ Selection criteria use actual evidence about requirements, risk, scope, schedule, team, and stakeholders.
☐ Scenario analysis includes a methodology choice, reasoning, tradeoff, and alternative.	☐ The team explains why at least two reasonable alternative methodologies were not selected.
☐ SDLC phases are mapped consistently to the selected methodology.	☐ Iterations, sprints, stages, or continuous workflow are represented accurately.
☐ The semester milestone plan includes dates/weeks, deliverables, and review or feedback points.	☐ Stakeholder involvement and change-management points are visible in the plan.
☐ DevOps and CI/CD are treated as complementary delivery practices, not as the primary methodology.	☐ The plan is feasible for the team size, available time, technical risks, and semester deadline.
☐ The written methodology-selection justification agrees with all worksheets and the final plan.	☐ Every group member can explain the methodology, milestone sequence, and major tradeoffs without reading a script.

16. Submission Requirements

- PDF of the final Development Methodology Plan in a clear, readable format.
- Completed Parts A-H and the 120-180 word methodology-selection justification.

- Export or screenshot of the semester milestone timeline, sprint/iteration plan, or task board used to support the final plan.
- Updated project GitHub repository or shared project workspace containing the methodology artifacts under a folder such as docs/methodology/.
- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA4_SystemName.
- Deliver a 5-7 minute group presentation using the final methodology plan as the main visual, followed by 2-3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own project conditions and planning decisions. AI tools may be used to clarify methodology concepts or review ideas, but the final comparison, selection criteria, methodology choice, SDLC mapping, milestone plan, and written justification must be checked, understood, and defended by the team. The instructor may ask any member to explain why a methodology was selected or trace how one feature will move through the planned process without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unclear reasoning, unequal participation, excessive reading, inconsistent milestone mapping, or inability to defend the selected methodology against reasonable alternatives.

Criterion	Evidence for Full Credit	Points
Methodology comparison accuracy	The comparison correctly distinguishes the major characteristics, strengths, limitations, and appropriate use of Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP.	10
Scenario analysis and tradeoff reasoning	Each scenario uses relevant project conditions to select a defensible method, explains why it fits, identifies a limitation, and considers a reasonable alternative or hybrid option.	8
Project-specific selection criteria	Requirement stability, risk, scope, schedule, team capability, stakeholder involvement, and need for change/feedback are evaluated using evidence from the actual project.	10
Selected methodology and written justification	The primary methodology is clearly named, correctly classified, supported by project evidence, compared with alternatives, and justified in a coherent 120-180 word explanation.	12
SDLC, iteration, sprint, or flow mapping	Planning, requirements, design, development, testing, deployment, review, and improvement are mapped accurately to the selected method and its stages, cycles, sprints, or continuous workflow.	12
Semester milestone plan	The schedule is realistic, project-specific, and aligned with the selected methodology; milestones contain measurable scope, deliverables, review/feedback points, and completion evidence.	12

DevOps and CI/CD complementary delivery plan	Version control, integration, testing, database change control, deployment, configuration, monitoring/feedback, and release/rollback practices appropriately support the selected methodology.	6
Group presentation and oral defense	The presentation is clear, organized, and within time; all members participate and correctly defend the methodology choice, project mapping, milestones, tradeoffs, and delivery practices.	10

Raw Score	Equivalent	Checked by	Date
/ 80			

18. Post-Laboratory Reflection

Which project selection criterion had the greatest influence on your methodology choice, and why?

The need for change and feedback had the greatest influence. Our project has known major features, but the user interface, classification workflow, and technical implementation may need refinement after adviser review and testing. Scrum gives us regular

sprint reviews where we can use that feedback to adjust the next sprint while keeping the semester scope controlled.

Which reasonable alternative methodology did your group reject, and what project condition made it less suitable?

We rejected Waterfall as the primary methodology because our project contains ML/image-processing uncertainty and may receive changes during development. A sequential process would make late changes more difficult to absorb. We can still use planned documentation and approval points, but the overall development needs short feedback cycles.

Where will the most important iteration, sprint review, phase gate, or workflow review occur in your semester plan?

The most important reviews will occur at the end of each two-week sprint, especially after Sprint 2 when the working Guso classification prototype is demonstrated. These reviews will allow the adviser and team to evaluate the classification workflow, identify defects or usability issues, and reprioritize the next sprint.

How will stakeholder feedback or requirement changes be handled without losing control of scope and schedule?

Feedback and change requests will be recorded as backlog items. The team will evaluate their priority, technical impact, and effect on

the semester schedule during backlog refinement. High-priority changes can be included in the next sprint, while changes that significantly expand scope will require team/adviser agreement before they are added.

Which DevOps or CI/CD practice should your team establish first, and what evidence will show that it is working?

The first practice should be a GitHub-based integration workflow with feature branches and automated checks before merging. Evidence that it works includes successful pull requests, review records, passing tests/build checks, and a clean main branch that can be deployed. This reduces integration problems as Flutter Web, FastAPI, PostgreSQL, and ML work are developed together.